

计算机图形学基础

简单了解: <https://chuquan.me/categories/%E5%9B%BE%E5%BD%A2%E5%AD%A6/>

清华课程: <https://cg.cs.tsinghua.edu.cn/course/resource.htm>

简单了解

1、线性代数、变换的基本知识

<https://chuquan.me/2024/01/15/foundation-of-computer-graphic-01/>: 包括有点、向量、矩阵, 点积、叉积等

<https://chuquan.me/2024/01/18/foundation-of-computer-graphic-02/>: 包括平移旋转齐次变换等基础变换知识

2、观测变换

我们将三维模型投影到二维屏幕的过程称之为 **观测变换** (Viewing Transformation)。

观测变换和我们平时拍照一样, 总体可以分成三个步骤:

- 摆放物体。在图形学中称为 **模型变换** (Model Transformation)
- 摆放相机。在图形学中称为 **视图变换** (View Transformation)
- 拍照。在图形学中称为 **投影变换** (Project Transformation)

根据这三个步骤的英文缩写, 观测变换也可以称为 **MVP 变换**。

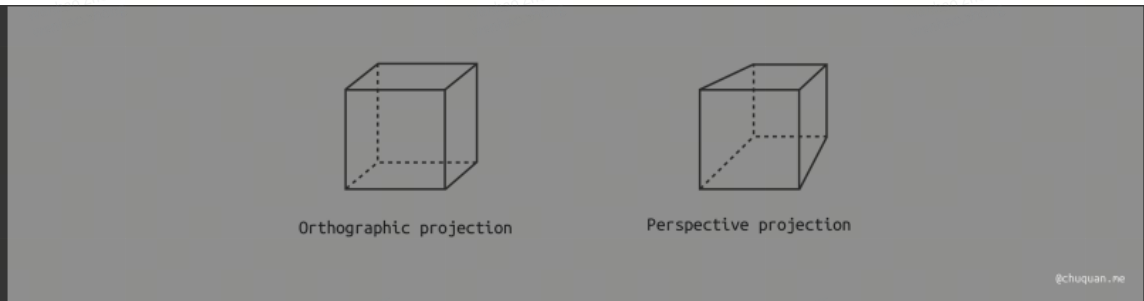
实际上就是得到相机的空间位姿, 然后将物体转换到相机坐标系下 (通过使用物体世界坐标乘上相机世界坐标的逆), 最后再进行3d-2d的投影

重点了解一下投影变换

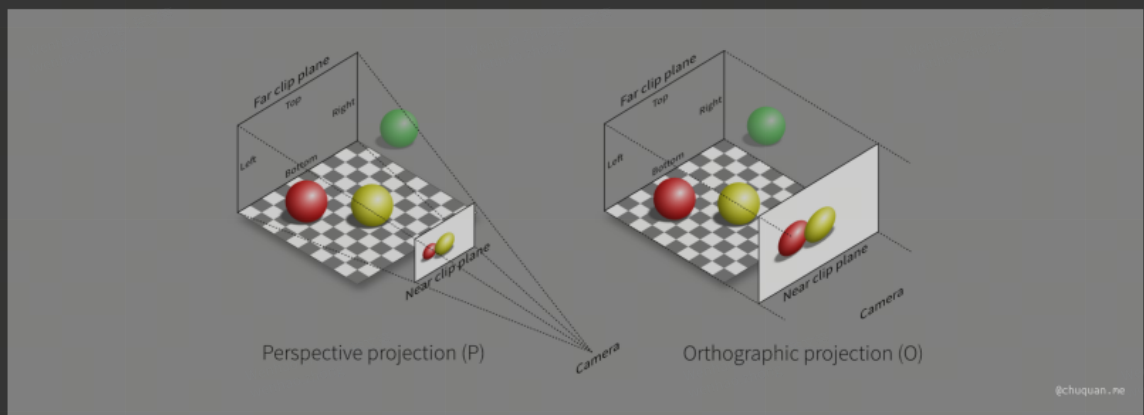
投影变换

投影变换本质上就是将 3D 模型投影到 2D 画布的过程, 具体可以分为两种:

- 正交投影 (Orthographic Projection)：一般用于工程制图软件，不具有近大远小的透视效果。
- 透视投影 (Perspective Projection)：一般用于游戏引擎、渲染引擎，模拟真实的效果。



事实上，正交投影可以认为是一种特殊的透视投影，即相机位于无限远的位置，如下所示。



正交投影

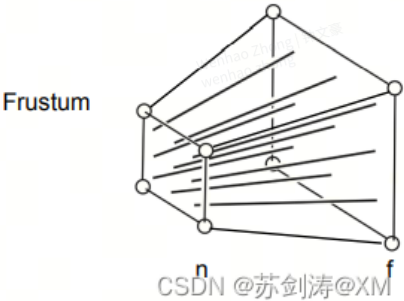
- 定义好一个三维空间中的观测区域，在正交投影的情况下应该是一个立方体，这个就是体现在图片中的部分
- 将观测区域平移放缩（不涉及旋转）到 $[1,1,1]$ 的标准空间中，这个变换就是正交投影变换，需要把所有物体点的坐标都乘以这个变换，然后就能够得到正交投影的结果

透视投影

使用和正交投影类似的方法，不过此时的观测区域是一个截断了的锥体，它多了一步压缩过程，也就是说，透视投影 = 压缩 + 正交投影

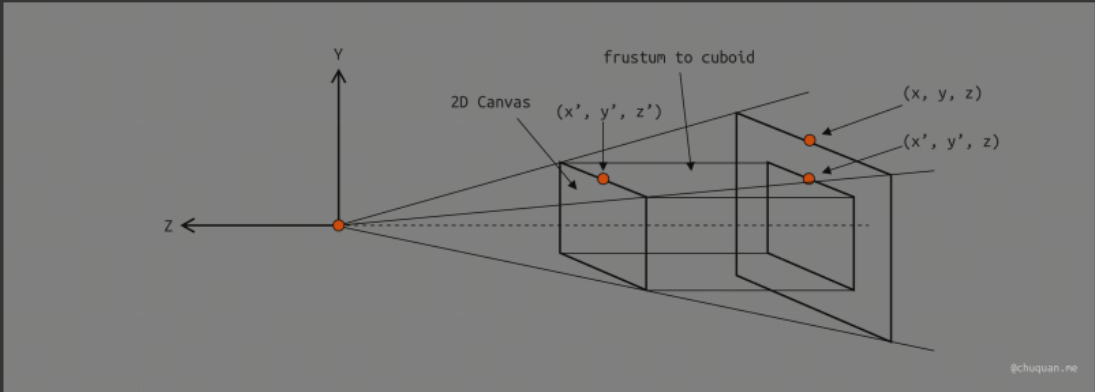
透视投影变换需要做的事。

首先我们观察下图中的几何体：



这是类似自然人的视野范围，或者叫做视锥体，它有一个比较小的近平面和一个比较大的远平面，这两个平面是近似等比关系。他们的中心点的 x, y 坐标都是0(观察点在原点)。

压缩的本质就是将透视投影的观测空间压缩成正交投影的观测空间，即将纺锥体转换成长方体。然后，透视投影就换转化成了正交投影了。

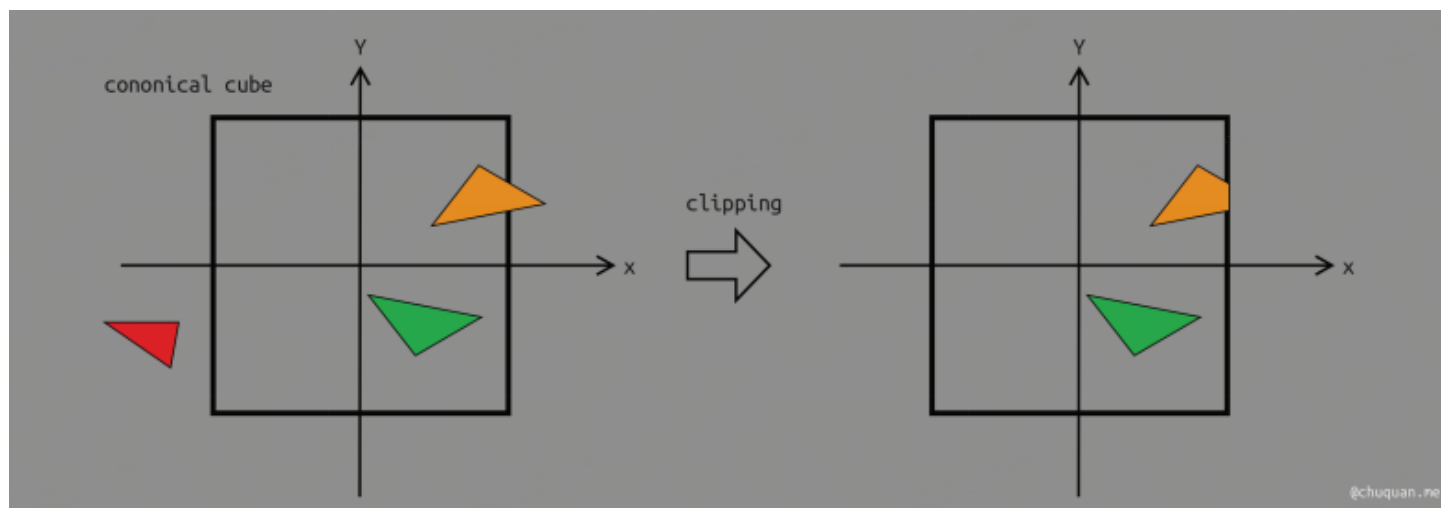


屏幕映射

完成了MVP变换之后，要开始将投影内容绘制到 2D 画布中，其中包含了裁剪和视口变换两个步骤。

裁剪

只有在观测空间的物体才是我们能够看见的，所以需要把空间中所有物体先转换，然后通过规范立方体对空间进行裁剪，只保留规范立方体内的物体。

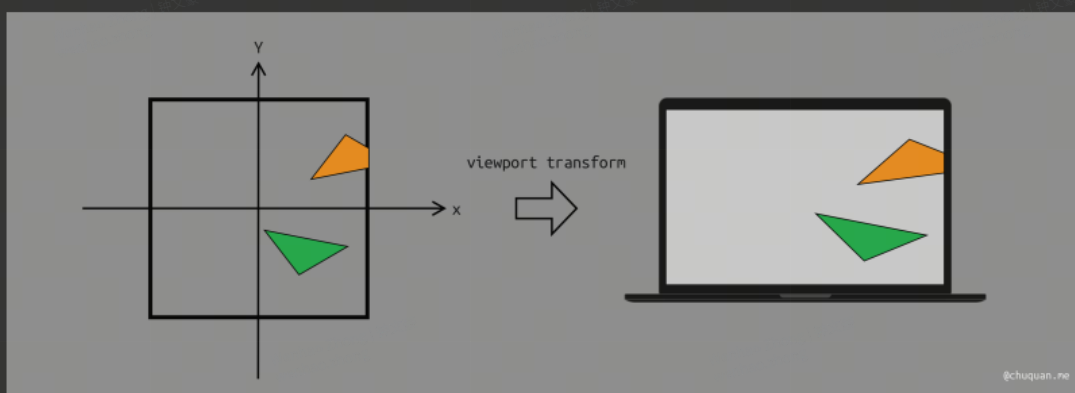


视口变换

就是根据屏幕的分辨率压缩图像

视口 (Viewport) 本质上就是我们所说的 2D 画布，即屏幕。我们知道屏幕有各种各样的分辨率，宽高比。为了处理这种情况，我们将 2D 画布抽象成一个 $[-1, 1]^2$ 的规范平面。然后通过视口变换将它映射到真正的视口中。

假设真实视口的宽度是 $width$ ，高度是 $height$ ，那么视口变换就是将 $[-1, 1]^2$ 的平面转换成 $[0, width] \times [0, height]$ 的平面。



对此，我们很容易求解变换矩阵，如下所示。

$$M_{viewport} = \begin{pmatrix} \frac{width}{2} & 0 & 0 & \frac{width}{2} \\ 0 & \frac{height}{2} & 0 & \frac{height}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

3、光栅化

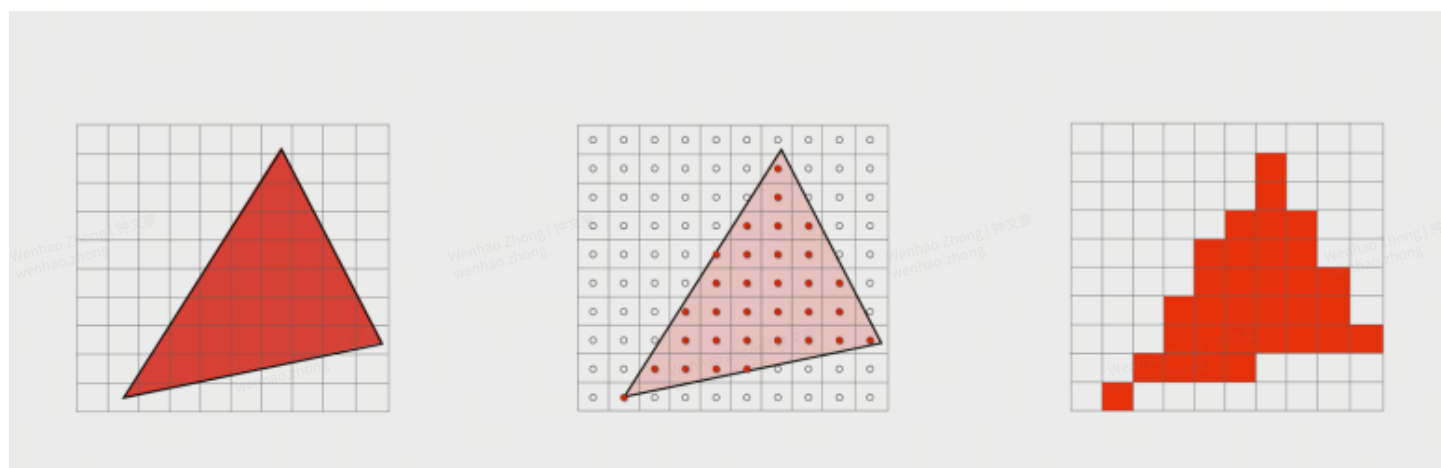
光栅化 (Rasterization) 中光栅 (Raster) 一词来源于德语，表示栅格的意思。我们现在用的显示设备基本上都是由像素点阵构成的栅格显示设备。因此，我们很容易理解光栅化的含义，即在栅格显示

设备上绘制图形。

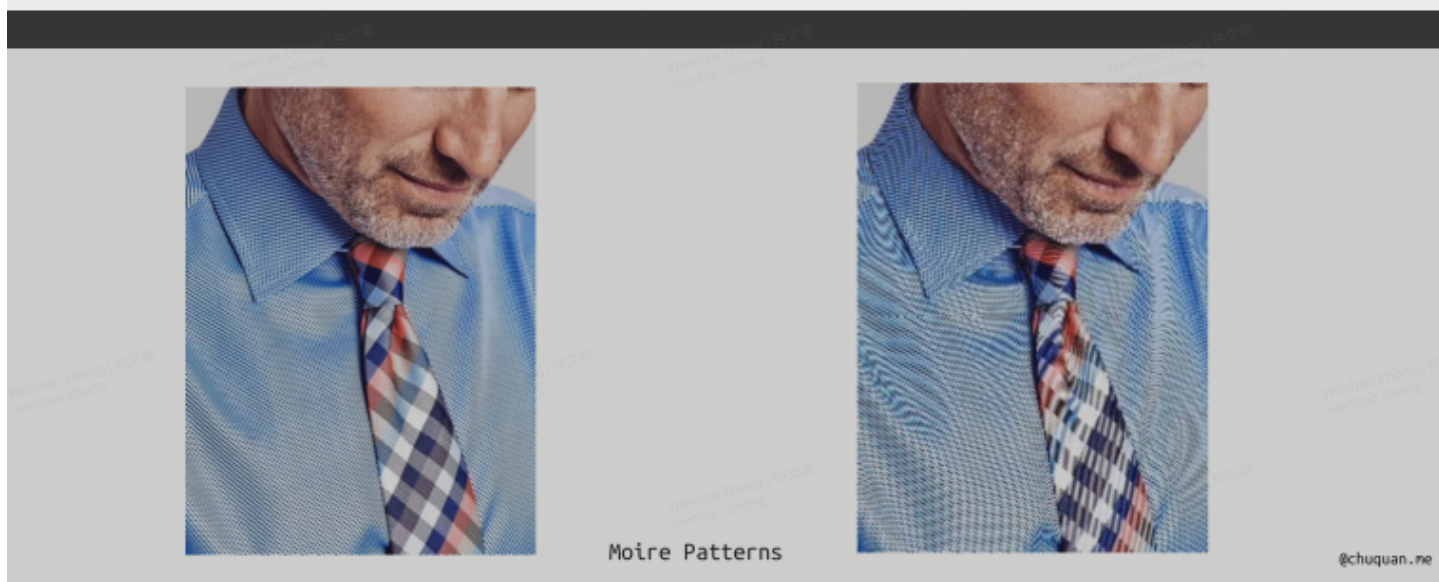
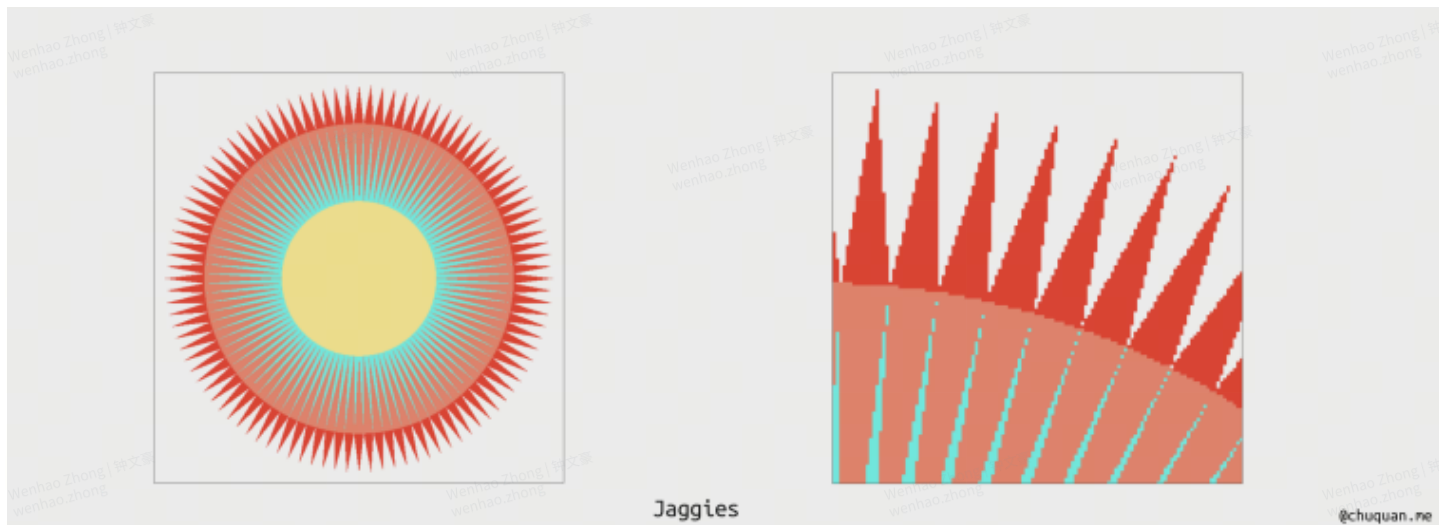
光栅化的**基本单元是三角形**，使用三角形的好处是：

- 三角形是最基本的多边形。
- 三角形具有平面性。
- 三角形可以明确定义内部和外部。我们可以通过向量叉积来判断，详见 [计算机图形学基础（1）——线性代数](#)。
- 任意多边形可以拆分成 N 个三角形。

在三角形区域内的点就是需要绘制的像素点，确认方法如图所示：

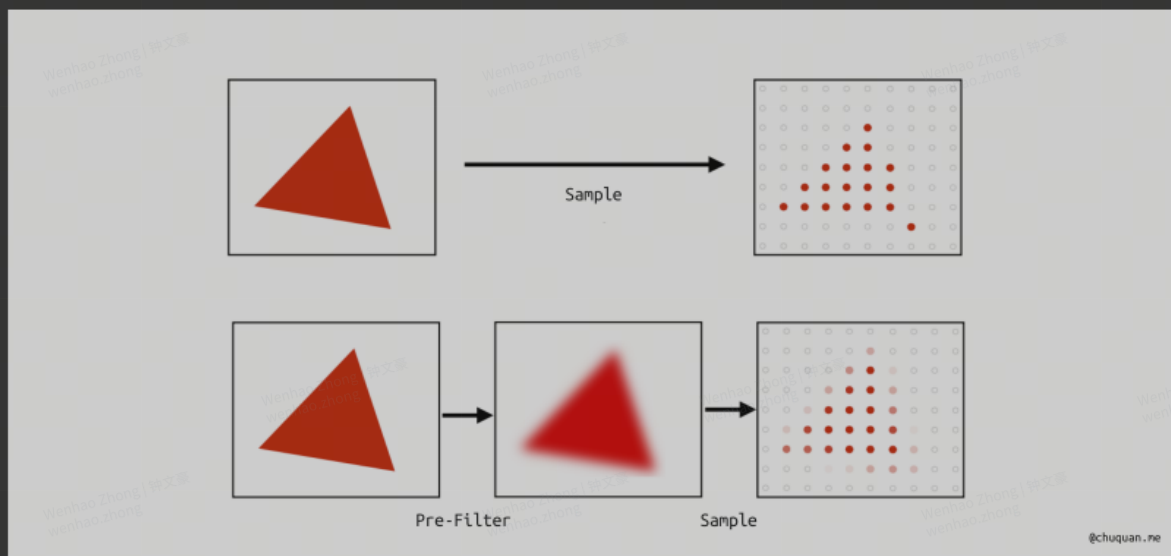


但是这样的情况会出现很多锯齿，这就是**走样问题**，光栅化要解决的核心问题就是走样问题，即**反走样（Antialiasing）**。

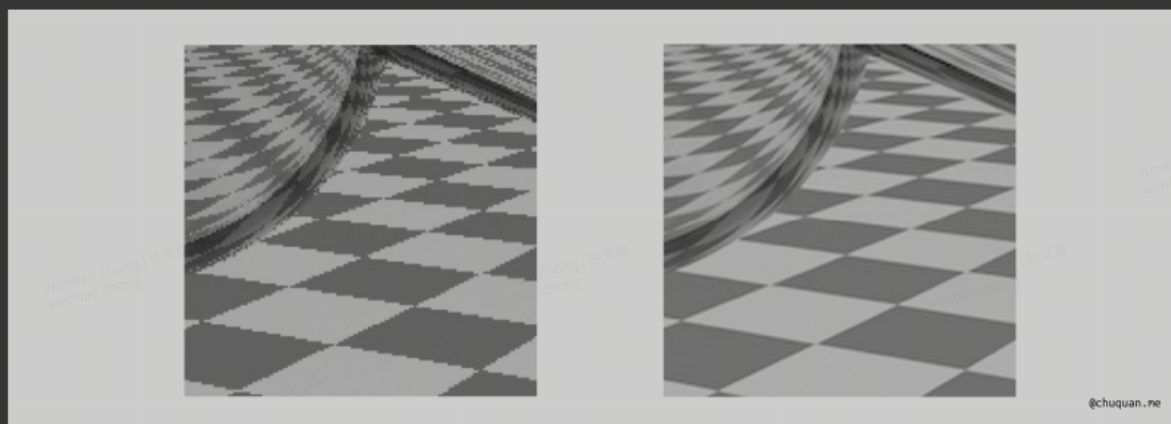


解决办法：最简单的解决办法就是先模糊（也就是滤波），后采样

下图所示，为「直接采样」和「先模糊，后采样」的流程对比图。



在具体实践中，通过这种方式能够有效解决光栅化中的锯齿问题，如下所示为反走样前后的效果对比图。



出现走样的根本原因是：采样频率小于真实频率，导致采样的数据无法还原出原始信号。滤波在反走样的过程中起到了重要作用，简单介绍图像处理中的滤波

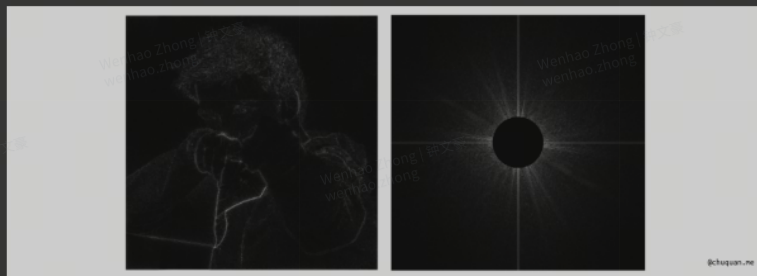
如下图所示，通过傅里叶变换将左侧的像素空间（空间域）变为右侧的频谱（频域）。对于二维信号，其频谱的表示如下：

- 高频部分代表细节、边缘、噪声
- 低频占据绝大多数能量，其中直流分量（零频）能量占比最大
- 频率分部具有中心对称性



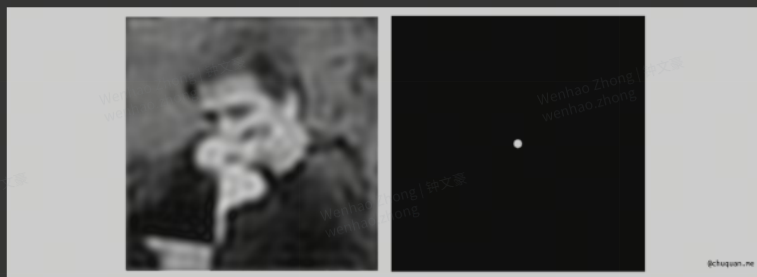
高通滤波

高通滤波（High-pass filter），保留高频信号。在图像中，轮廓的边缘会发生剧烈变化，属于高频信号。经过高通滤波后，图像只会保留一些轮廓信息，如下图所示。



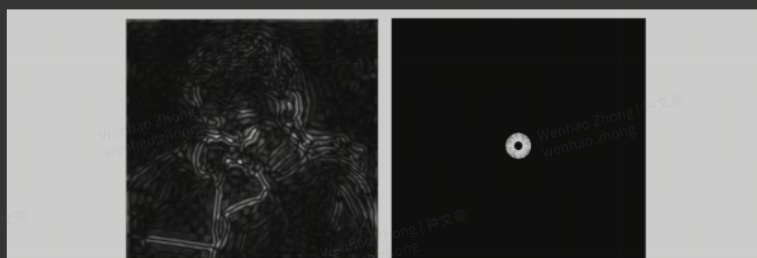
低通滤波

低通滤波（Low-pass filter），保留低频信号。在图像中，颜色变化平缓的区域属于低频信号。经过低通滤波后，图像会抹去轮廓信息，如下图所示。模糊处理基于低通滤波实现的。



带通滤波

带通滤波（Band-pass filter），顾名思义，只保留一部分频率范围内的信号。对图像滤波后的效果取决于带通滤波所选择的频率范围。下图所示，为两种不同频率范围的带通滤波。



使用卷积来实现滤波的效果

实现反走样的方法主要就是围绕两个角度来实现：

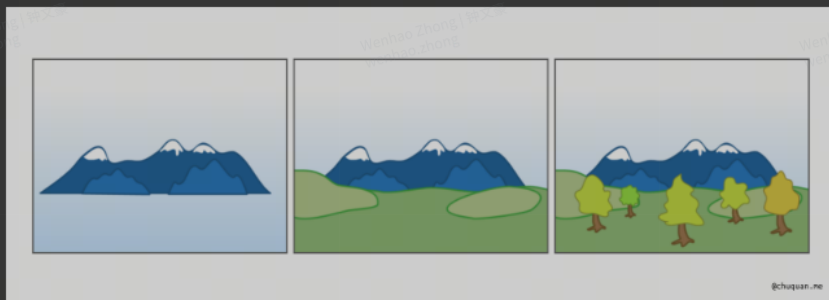
- 提高采样频率。如：超采样技术（Supersampling）、多重采样抗锯齿（MSAA）、超分辨率
- 过滤高频信号。如：先模糊后采样（Pre-Filter）

遮挡与可见

在 3D 空间中，三角形之间存在着前后遮挡关系，那么三角形绘制的先后顺序应该是什么样的呢？

画家算法：远的先绘制，近的后绘制

画家算法，顾名思义，按照画家绘画时的先后顺序来执行，远的物体先绘制，近的物体后绘制，如下图所示。



虽然画家算法适用于绝大多数场景，但是在某些场景下它仍然无法解决可见性问题。如下图所示，三个相互嵌套的三角形，使用画家算法则无法对三角形进行排序，因此无法准确实现光栅化。



深度缓冲算法

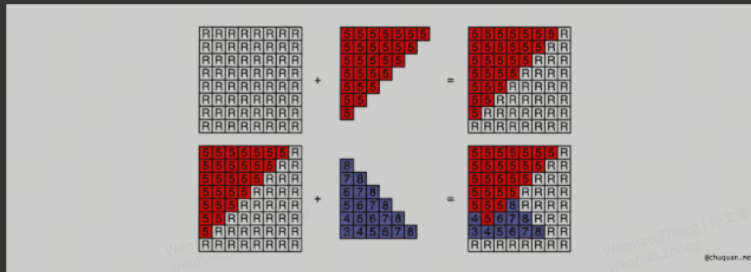
那么上述问题该如何解决呢？于是出现了 **深度缓冲算法**（Z-Buffer Algorithm），其基本原理是：

- 光栅化采用两个缓冲区
 - 原有的 **帧缓冲区**（Frame Buffer）存储每个像素颜色值
 - 附加的 **深度缓冲区**（Z-Buffer）存储每个像素深度值
- 深度缓冲区存储每个像素当前的 **最小深度值**（Z-Value）

如下所示，为深度缓冲算法的伪代码实现。注意：我们会初始化深度值为无穷大。

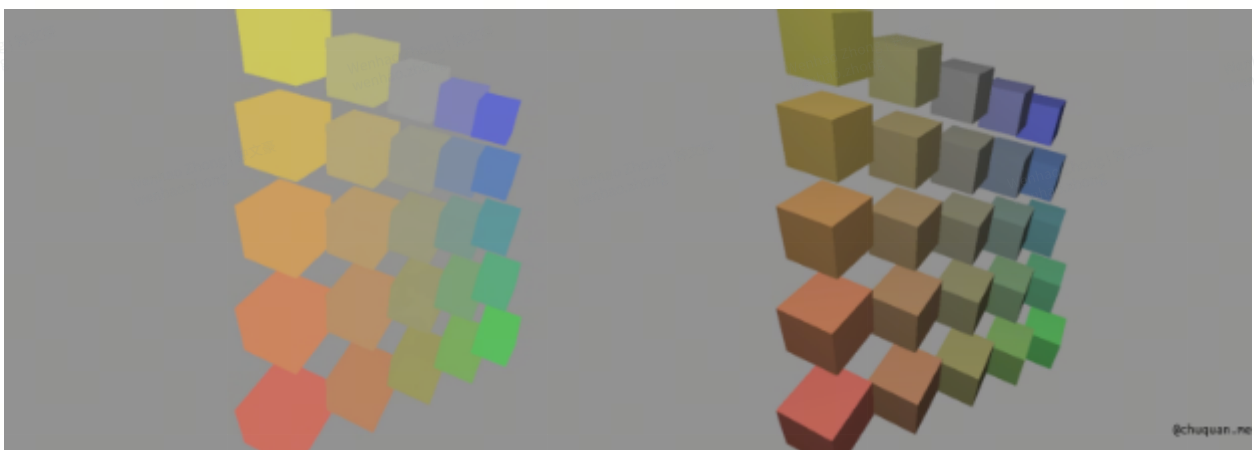
```
1 for (each triangle T) {  
2     for (each sample (x, y, z) in T) {  
3         if (z < zbuffer[x, y]) {           // 处理深度更小的采样点  
4             framebuffer[x, y] = rgb;       // 更新颜色值  
5             zbuffer[x, y] = z;             // 更新深度值  
6         }  
7     }  
8 }
```

下图所示，使用深度缓冲算法光栅化两个三角形的示意图。当光栅化红色三角形时，我们遍历红色三角形的每一个像素的深度值，并与当前深度值进行比较。由于当前深度值均为无穷大，所以红色三角形的每一个像素都可以绘制。当光栅化蓝色三角形时，同样会遍历蓝色三角形每一个像素的深度值，并与当前深度值变换，深度值大于当前深度值，则不绘制；否则，绘制并更新当前深度值。



4、着色

仅仅实现光栅化，还不足以让渲染结果具有真实感，如下图左部所示。我们希望能够模拟光线所带来的的明暗效果，如下图右部所示。



在计算机图形学中，**着色**（Shading）就是通过计算来决定三维模型表面每个像素的颜色和亮度的过程。本质而言，着色就是 **对不同物体应用不同材质**。

着色局部性

具体分析着色时，我们会分析光线照射到物体表面的每一个点，也称 **着色点**（Shading Point）。对于每个着色点，我们将其视为一个微平面（或称单位平面），由此我们可以构建法线。整体而言，着色的最终结果受以下几种输入影响，分别是：

- 观测方向 v
- 表面法向 n
- 光线方向 l
- 表面参数，如：颜色、材质。

注意：对于着色过程，我们只考虑光照对于物体表面的影响，而不考虑其他物体的阴影对本物体产生的影响。

